

Reviewer Pack — Minimal Order of Brauer Groups and Communication Complexity ...

Entry d04adc422e8c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 00:14:13 UTC

Minimal Order of Brauer Groups and Communication Complexity Rank Variance

Entry ID: d04adc422e8c **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 00:14:13 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Brauer Group Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every boolean function f , the minimal order of its associated finite group G_f , determined by its brauer group, is linearly correlated with the variance of the communication complexity rank of f on all n -input Boolean functions, such that $\log |G_f| = \Theta(\text{Var}(\text{Rank}(f)))$ for large n .

Rationale (proposer's reasoning):

Brauer groups capture non-commutative algebraic structure in a function's representation, which may correlate with the complexity of distributing information among parties. A direct mapping from Brauer group order to communication complexity rank variance could expose new structural insights into both fields.

Taxonomy category: BrauerGroupCommunicationComplexityRank (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 79ce24a3a8437bab

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given boolean function f , if the correlation coefficient between $\log |G_f|$ and $\text{Var}(\text{Rank}(f))$ is ≥ 0.8 across at least 24 out of 30 independent random seeds, then support for the conjecture is provided.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal order Brauer groups AND Communication Complexity rank variance
- Brauer group theory IN ALGEBRA AND Matrix Rank in Communication Complexity - log |G_f|
correlation with Var(Rank(f)) Boolean functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank(f):
        n = len(f)
        ranks = []
        for i in range(2**n):
            rank = 0
            for j in range(2**n):
                if f[i] == f[j]:
                    rank += 1
            ranks.append(rank)
        return sum(ranks) / (2**n * n)

    def brauer_group_order(f):
        # Placeholder for Brauer group order calculation
        # This is a dummy implementation and should be replaced with actual logic
        return len(f)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        f = generate_boolean_function(n)
        order = brauer_group_order(f)
        rank = communication_complexity_rank(f)
        results.append((order, rank))
```

```

if not results:
    return {
        "metric_name": "correlation",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

orders = [r[0] for r in results]
ranks = [r[1] for r in results]
mean_order = sum(orders) / len(orders)
mean_rank = sum(ranks) / len(ranks)
variance_rank = sum((x - mean_rank)**2 for x in ranks) / len(ranks)

correlation = (sum((orders[i] - mean_order) * (ranks[i] - mean_rank) for i in range(len(orders))) *
                math.sqrt(sum((orders[i] - mean_order)**2 for i in range(len(orders))) *
                             sum((ranks[i] - mean_rank)**2 for i in range(len(ranks))))))

return {
    "metric_name": "correlation",
    "metric_value": correlation,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": abs(correlation) >= 0.8 and len(results) >= 24,
    "counterexample": "" if abs(correlation) >= 0.8 else "correlation < 0.8"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation < 0.8\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_45ea5583.py", line 87, in <module>
    result = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_45ea5583.py", line 46, in run_trial
    rank = communication_complexity_rank(f)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_45ea5583.py", line 30, in communication
    if f[i] == f[j]:
        ~^^
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its intended task of calculating the correlation coefficients. | next: Re-run the test with proper error handling to ensure that it completes without crashing and produces the necessary data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13539
2	preregistration	ollama_remote	glm4:latest	0	0	9332
3	novelty	ollama_remote	glm4:latest	0	0	10089
4	novelty	ollama_remote	glm4:latest	0	0	9125
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11851
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8204
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10379
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10823
9	judge	ollama_remote	glm4:latest	0	0	20482

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 103824 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d04adc422e8c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d04adc422e8c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d04adc422e8c.tar.gz` (if generated)